

ЛАБОРАТОРИ АЖИЛ №4 ОНОО - 4

I ХЭСЭГ

Зорилго: Энэхүү лабораторийн ажлын зорилго нь олон зангилаанаас бүрдэх хувиарлагдсан системийг загварчлан хэрэгжүүлэх замаар node-ууд хооронд мэдээлэл солилцох механизмыг ойлгох, хэрэглэгчийн ачааллыг олон серверт хуваарилан тэнцвэржүүлэх (load balancing) аргачлалыг турших, мөн системийн аль нэг хэсэг доголдсон тохиолдолд үйлчилгээ тасалдахгүй үргэлжлэх (fault tolerance) боломжийг судлахад оршино.

Холболтын бүтэц: Систем нь хоёр сервертэй байна. Үүнийг А, В гэж нэрлэе. Харин серверт холбогдох клиентүүдийг 1, 2, 3 г.м дугаарлана. Хэрэв хэрэглэгчид дараах байдлаар холбогдвол

Клиент 1 - > Сервер А, Клиент 2 - > Сервер А,

Клиент 3 - > Сервер В, Клиент 4 - > Сервер В

Сервер А болон Сервер В нь хоорондоо мэдээлэл солилцож, өөр серверт холбогдсон клиентүүдэд мессежийг дамжуулах үүрэгтэй байна. Өөрөөр хэлбэл, аль нэг сервер дээр ирсэн мэдээлэл нөгөө серверт дамжиж, бүх клиентүүдэд хүрэх боломжтой байх ёстой.

1. Node.js ашиглан Сервер А(8001) болон Сервер В(8002) үүсгэх ба тус бүр өөр порт дээр ажиллана. Серверийг Socket.IO ашиглан клиентүүдтай холбох ба сервер нь клиентээс message хүлээн авах холбогдсон клиентүүдийн жагсаалт хадгалах үүрэгтэй.
2. Сервер А болон Сервер В хооронд холболт үүсгэх ба нэг сервер дээр ирсэн message-ийг нөгөө сервер рүү дамжуулна. Клиент - > Сервэр А - > Сервер В - > Клиент
3. Клиентүүдийг серверт хуваарилахдаа ачааллыг тэнцвэржүүлнэ (Load blancing). Жишээ нь тэгш дугаартай хэрэглэгч клиентийг А, сондгой дугаартай хэрэглэгч клиентийг В сервертэй тус тус холбоно.
4. Систем нь алдаанд тэсвэртэй байж (Fault tolerance) тодорхой гүйцэтгэлтэйгээр ажилладаг байх ёстой. Сервер А-г унтраахад Сервер В дээрх клиентүүд хоорондоо мессэж дамжуулах боломжтой байна. А зогссон үед шинэ хүсэлтүүдийг бүгдийг В рүү чиглүүлнэ.

II хэсэг:

Зорилго: Тус хэсгээр саваа металын дулааны тархалтыг тооцоолно. Савааг шугаман (1D) массиваар илэрхийлэх ба массивын анхы утга эхний удаад бүгд 0(зүүнээс бусад) байна. MPI ашиглах гол зорилго нь процессууд хооронд хүрээний өгөгдлийг солилцох шаардлага үүснэ.





1. Хуваарилалт: Урт савааг нийт N хэсэгт хуваагаад, хэсэг бүрийг MPI процесс тус бүрт (Rank) тэнцүүгээр хуваарилна. нийт цэгийн тоо $N = 4096$, нийт процесс $size = 4$, процесс бүр $local\ n = N / size = 1024$ цэг хариуцана.
2. Харилцаа: Процесс бүр өөрийн хэсгийн (internal points) дулааныг тооцоолно. Харин дараагийн цаг хугацааны алхамд шилжихийн тулд, тухайн процессийн баруун захын утга баруун хөрш-дөө, зүүн захын утга зүүн хөрш-дөө хэрэгтэй болно. ӨХ зүүн хөршөөс нэг, баруун хөршөөс нэг boundary утга авч өөрийн зүүн, баруун захын утгыг хөрш процесс руу илгээнэ.
3. Нийт процессын тоо 4 (mpirun -n 4 ...)
4. Нийт савааны урт 4096 цэг.
5. Эхлэх дулаан 100 (зүүн талын утга) Rank 0-ийн `current_temp[0] = 100.0`, Бусад бүх цэгийн анхны температур: Цаг хугацааны алхамын тоо 100, 1000 турших
6. Дулаан дамжуулалтын коэффициент $C = 0.25$
7. Дулааны тархалтыг хөрш цэгүүдийн хоорондох температуудын зөрүүгээр тодорхойл (Finite Difference Method)
`next_temp[i] = current_temp[i] + C * (current_temp[i - 1] - 2.0 * current_temp[i] + current_temp[i + 1]);`

Алгоритм: Эхний температурыг оноож дараах алхмыг давтана:

- хөрш процессуудтай boundary утга солилцох
- шинэ температурыг бодох
- массивыг шинэчлэх
- Эцсийн температурыг гаргах

Үр дүн: Тооцоолол дууссаны дараа савааны дагуух температурын тархалтыг нэгтгэн гаргаж, график хэлбэрээр дүрслэн харуулна. Үүнд:

- Процесс бүр өөрийн тооцоолсон температурыг хадгална
- Бүх процессын өгөгдлийг Rank 0 процесс дээр нэгтгэнэ. Rank 0 процесс нь дараах форматтай файлыг үүсгэнэ.
- Үр дүнг графикаар дүрслэхдээ: X тэнхлэг нь байрлал (position) Y тэнхлэг нь температур (temperature) байна.

Output.csv файлын бүтэц:

position, temperature

0, 100.0

1, 99.8

2, 99.6

...

4095, 0.0